COMP 551 – Training and Evaluation Practices

Anto Catanzaro Ethan Gat Marc Saliba
Group 22, McGill University

October 26, 2026

Abstract

The project aims to implement a logistic regression model to predict whether or not an unseen email is spam. The model is trained with stochastic gradient descent, via hyperparameters that need tuning via K-fold cross-validation. L2 regularization is then used to reduce overfitting. Results suggest that learning rate and batch size have the greatest impact on CV performance. Cross-validation selected an intermediate $\lambda = 0.01$, achieving 90% test accuracy. Fitting a logistic regression model with L1 regularization instead, we found that stronger regularization strength resulted in a sparser model at the expense of model performance.

1 Introduction

The UCI Spambase dataset contains 4,601 email instances, each described by 57 predictive features and a binary label indicating whether each email is spam. Of the 57 features, 48 are continuous variables representing the percentage of words in an email matching specific terms. Another six continuous variables represent the percentage of characters matching particular characters. The remaining three continuous variables describe the average, longest, and total lengths of uninterrupted runs of capital letters. The target binary label has spam $\{0, 1\}$, where 1 indicates spam and 0 indicates non-spam. The objective of the project is to develop, train, and evaluate a logistic regression model for spam classification on unseen emails. Logistic regression is a discriminative classifier that estimates the probability of an instance belonging to a class, weighted by its features. For an input email ($\mathbf{x}$), the probability that an email is spam is given by

$$P(Y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x})}} \tag{1}$$

2 Methods

2.1 Data and Preprocessing

We use a 5% train, 95% test split. Each feature is standardized with mean and standard deviation on training set. The train statistics are used to standardize the test set.

2.2 SGD Logistic Regression

To implement SGD logistic regression, we created two separate classes: `StochasticGradientDescent` and `LogisticRegression`.

SGD is split into two classes. `StochasticGradientDescent` computes mini-batch SGD with input batch size, learning rate, and number of epochs. Data is reshuffled at the start of each epoch. `LogisticRegression` accepts regularization term λ and initialization scale. Its `fit` method builds the gradient, passing to the optimizer, and `predict` returns a probability of positive class occurring.

L2 regularization convention We used the following loss and resulting gradient.

$$\mathcal{L}_{reg}(\mathbf{w}, b) = \sum_{i=1}^{N} -y^{(i)} \log(\hat{y}^{(i)}) - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) + \tfrac{\lambda}{2} \|\mathbf{w}\|_2^2 \tag{2}$$

$$\nabla \mathcal{L}_{reg}(\mathbf{w}, b) = \sum_{i=1}^{N} x^{(i)} (\hat{y}^{(i)} - y^{(i)}) + \lambda \mathbf{w} \tag{3}$$

2.3 K-Fold Cross-Validation

We use $K = 5$ fold cross-validation on the 5% training split. The training indices are shuffled and divided into K nearly equal fold. Each fold becomes the validation set once while the model is trained on the other $K - 1$. To avoid leaking information, the same folds are reused for every hyperparameter setting and the standardization describing statistics are recalculated. We return the mean and standard deviation of cross-entropy and accuracy across the folds, and the held-out 95% test set is never used during tuning.

3 Results

3.1 Task 1: Logistic Regression with SGD

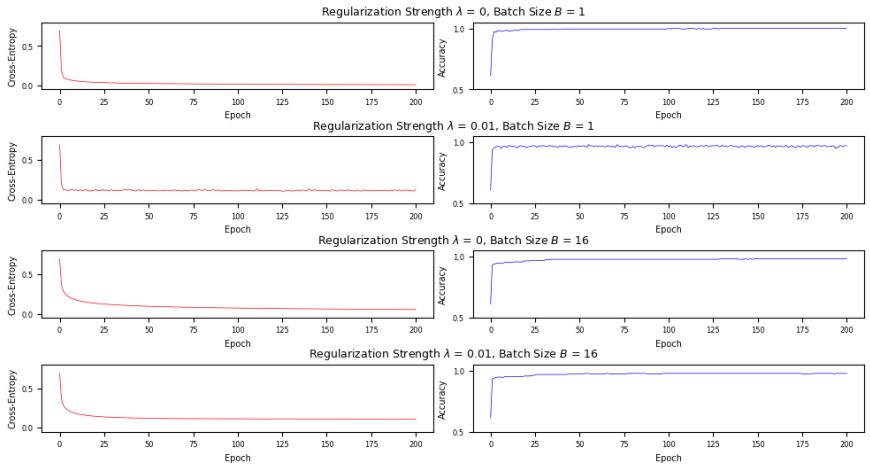


Figure 1: Training cross-entropy vs. epoch, with and without L2 regularization and with and without mini-batch.

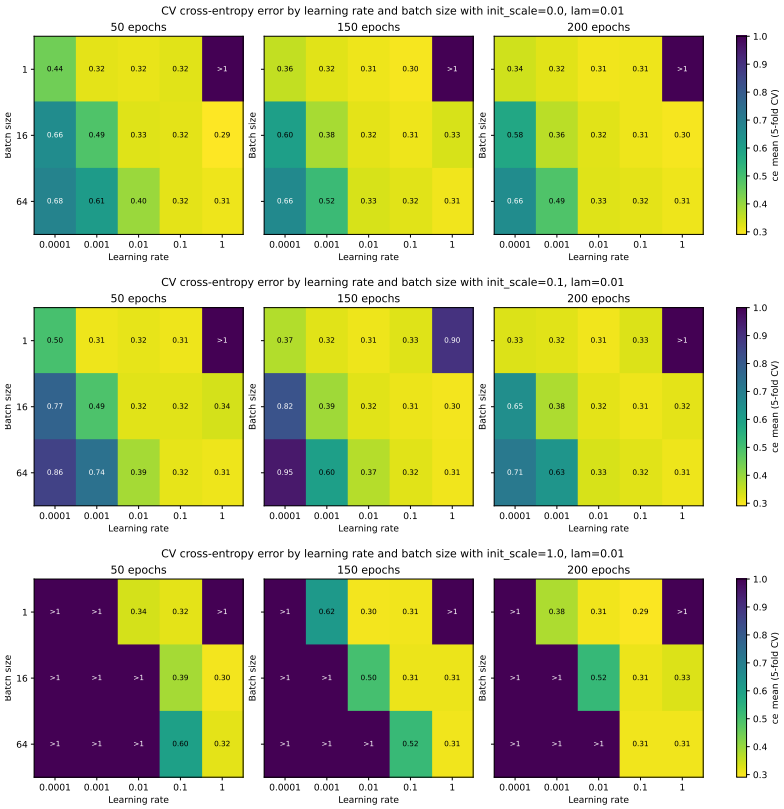
Adding L2 regularization appears to decreases stability, while using a mini-batch increases stability. Additionally, conventional SGD appears to converge faster than mini-batch SGD.

3.2 Task 2: Hyperparameter Tuning with K-Fold CV

The hyperparameters to tune are **batch size**, **learning rate**, **number of epochs**, and **initialization scale** of weights.

Table 1: Top-5 CV results for the hyperparameter grid.

B	lr	epochs	init_scale	Mean CE	Std CE	Mean Acc.	Std Acc.
16	1.0	50	0.0	0.290729	0.161530	0.926087	0.057680
1	0.1	200	1.00	0.294069	0.163610	0.921739	0.052535
1	0.1	150	0.0	0.299541	0.170064	0.926087	0.057680
16	1.0	150	0.01	0.300514	0.171546	0.921739	0.059296
16	1.0	200	0.0	0.301773	0.165809	0.921739	0.059296



Analysis of Table & Heatmaps Learning rate and batch had the largest impact, varying the most, with approximately equal effect on cross-entropy mean. Intuitively they also tend to counteract, as a small learning rate underfits for large batch sizes, meanwhile a large learning rate enables a small batch size to diverge. The number of epochs mattered only when the learning rate was small, and initialization scale only impacted (and negatively) when it was too large (= 1). To accomodate these findings and evaluate the best configuration, we created a function to extract from the table of permutations on the following criteria.

Figure 2: CV cross-entropy by learning rate and batch size.

To pick the best configuration of hyperparameters, we employ the one-standard-error rule.

1. Keep runs with one init_scale, since we observe it did not matter much
2. Average cross-entropy mean over epochs, giving a single estimate per (B, lr) cell
3. Score a cell by averaging it between its neighbours learning rates
4. Any cell within one SE is a tie
5. Finally we pick the largest batch size, then smallest learning rate (the fastest while also most convergent solution)

This is the one-standard-error rule, with additional picking logic. We are effectively treating configurations whose CE mean is within $\sigma/\sqrt{K}$ (standard error, $\sqrt{\sigma^2/k}$) as a tie, and picking preferred configurations based on the above criteria. Running the search gives the following configuration:

Best Configuration: $B = 64$, $\eta = 0.1$, 200 epochs, zero initialization.

3.3 Task 3: Bias–Variance Trade-Off via a λ Sweep

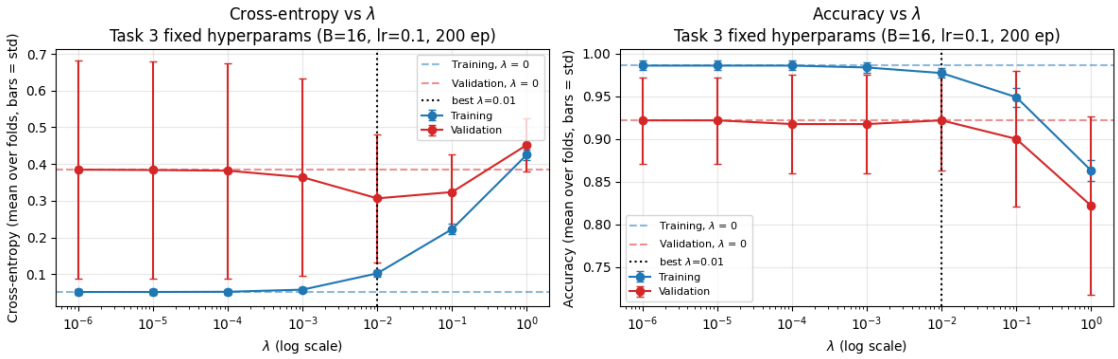


Figure 3: Fixed-λ Sweep. (B=16, lr=0.1, 200 epochs); CE (left panel) & Accuracy (right panel)

From CV, the selected $\lambda = 0.01$. Small λ ($\leq$ 1e-4) training CE is very low, but validation CE is much higher, so there is a large gap. This is overfitting. This happens because, with almost no penalty, the weights grow and fit noise causing the fit to be heavily dependent on the sample (high variance). At $\lambda = 1$, training CE rises over 0.4 and training accuracy falls, so the model underfits. This shows high bias. This is because, with a large penalty, the weights shrink towards 0, so the model is too strongly constrained to properly capture the pattern from the training data. Furthermore, the validation CE curve has a U-shape because as λ increases, variance falls and bias rises, so validation error is at its lowest ($\lambda = 0.01$) when variance and bias are balanced.

Comparing the fixed hyperparameters and the best hyperparameters from Task 2(See Figure 7 in Appendix A), both select $\lambda = 0.01$ and their curves converge for $\lambda \geq 10^{-2}$ (See figure 8 in Appendix A which graphs both). At the selected $\lambda = 0.01$, the gaps are about equal. The difference lies at small λ where the best hyperparameters config overfits less with a smaller generalization gap.

Table 2: Final test performance.

λ	Test cross-entropy	Test accuracy
0.01	0.3759	0.9009

Optionally, we studied the regularization effect of dataset size (See figure 9 in Appendix A). As dataset size increases, the generalization gap becomes smaller. This is because overfitting is expected to reduce when there is lower variance.

3.4 Task 4: L1-Regularized Logistic Regression and the Regularization Path

A regularization path shows how a model's weights change as regularization strength decreases.

As shown in figure 5, L1 regularization induces sparsity, with more coefficients becoming non-zero as regularization strength decreases. The first five features to become non-zero as regularization strength decreases are, in order, `char_freq_$`, `char_freq_!`, `word_freq_remove`, `word_freq_money`, and `word_freq_your`, indicating that these are the most important features for spam detection.

As shown in figure 6, as regularization strength decreases, model performance increases. However, as shown in the figure 5, sparsity, and thus interpretability, decrease. Therefore, we must compromise between interpretability and performance. We can balance these two factors by choosing the simplest model within one standard deviation of the model with the lowest validation loss, as discussed in class.

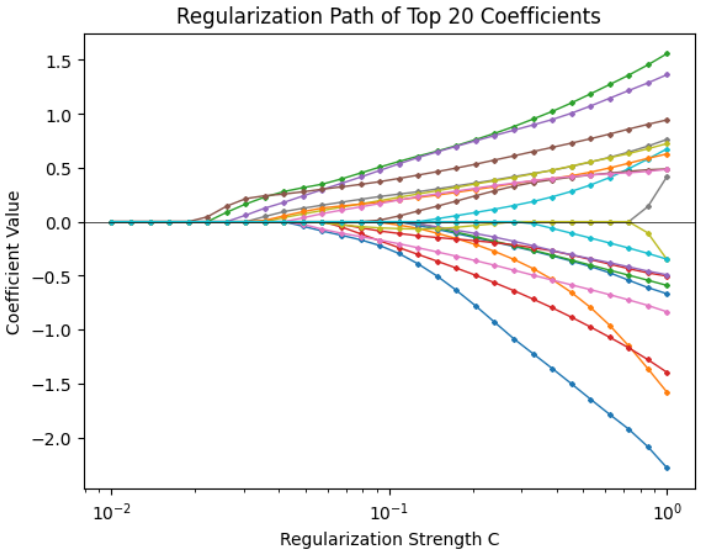


Figure 4: Regularization path of the top 20 coefficients by magnitude at C = 1

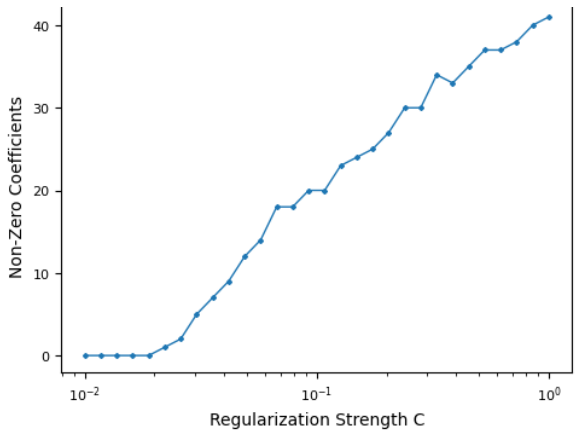


Figure 5: Number of non-zero coefficients vs. regularization strength C

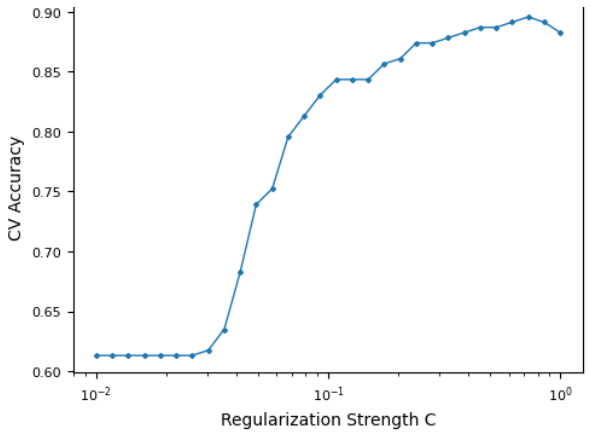


Figure 6: Mean cross-validation accuracy vs. regularization strength C

4 Discussion and Conclusion

Logistic regression trained using mini-batch SGD classified the dataset well with only 5% of the data used for training, where it reached a test accuracy of 0.901 and a test cross-entropy of 0.376 at the CV-selected $\lambda = 0.01$ (See Table 2). The λ demonstrated the bias-variance tradeoff where small λ overfit and large λ underfit. With CV, an intermediate λ was found to be optimal at the minimum of the validation error U-shape. Regarding L2 and L1 regularization, L2 shrinks all weights smoothly and keeps all features whereas L1 can set weights to zero, giving a sparser,

more interpretable model at the cost of some accuracy. The main limitations are that the small training set makes CV estimates noisy, with a standard deviation large relative to the gaps between the configurations, and that λ was not tuned together with the other hyperparameters. Future steps could use repeated CV, joint random search to tune hyperparameters, and metrics such as precision and recall, which matter for spam filtering. Overall, SGD logistic regression with CV-tuned regularization gives an effective spam classifier, and the choice between L2 and L1 comes down to accuracy versus interpretability.

Statement of Contributions The group collaborated on all parts of the project to meet requirements, working together.

Statement on the Use of LLMs LLMs were not used in the completion of this project.

References

[1] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt. Spambase. UCI Machine Learning Repository, 1999. https://doi.org/10.24432/C53G6X.

[2] I. Androutsopoulos, J. Koutsias, K. V. Chandrinos, G. Paliouras, and C. D. Spyropoulos. An evaluation of Naive Bayesian anti-spam filtering. *arXiv preprint* cs/0006013, 2000. https://arxiv.org/abs/cs/0006013

A Additional Figures

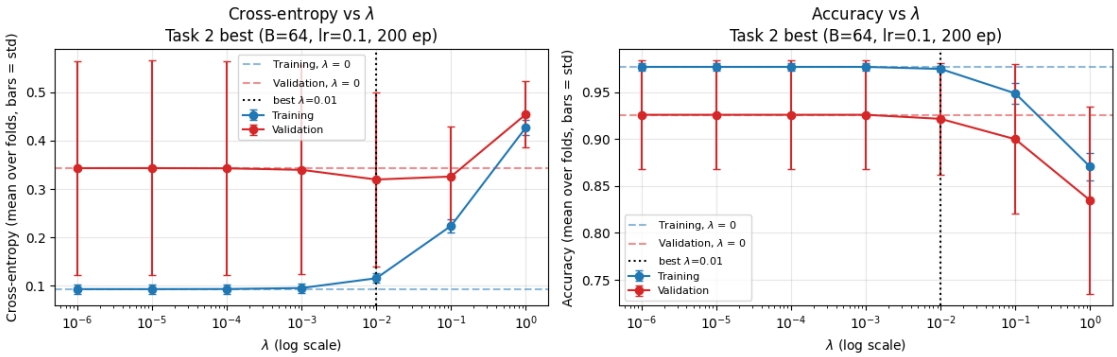


Figure 7: Best-λ Sweep. (B=64, lr=0.1, 200 epochs); CE (left panel) & Accuracy (right panel)

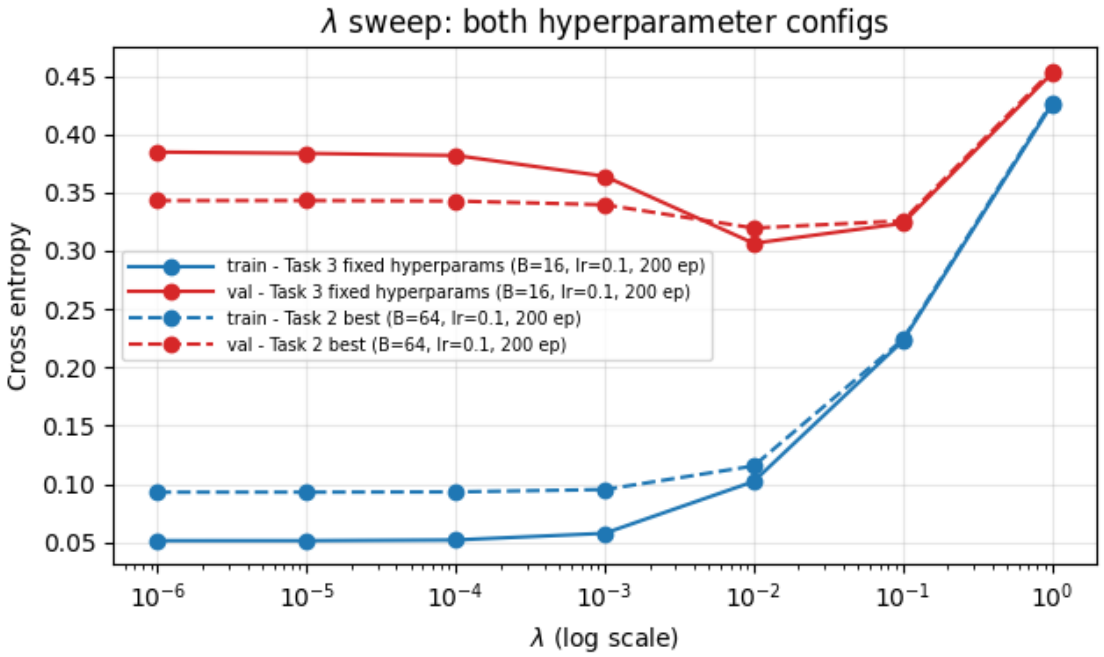


Figure 8: Both Configs - λ sweep.

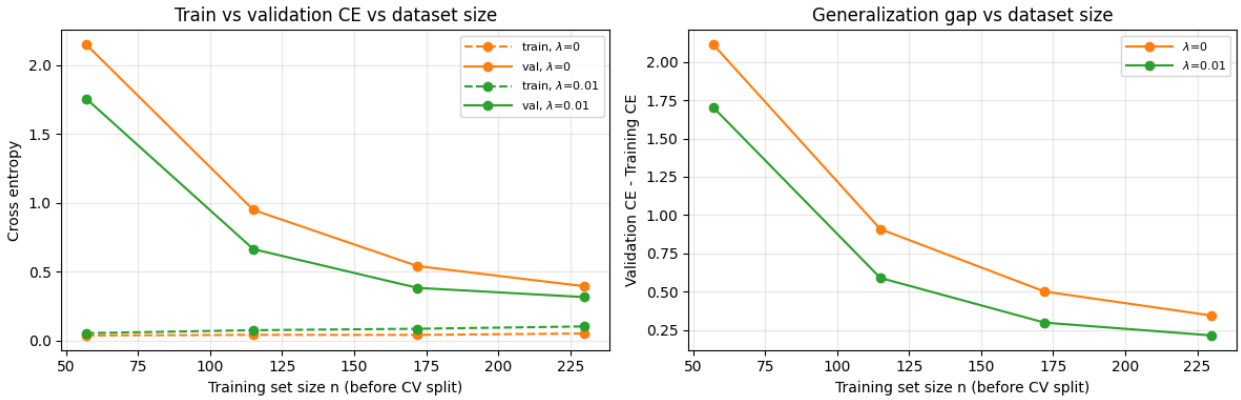


Figure 9: Regularization Effect of Dataset Size